

resiliencia-ante-fallos-parciales

- Clase 07 — Resiliencia ante fallos parciales
 - 1. Contexto: del fallo local a la falla en cascada
 - 2. Presupuestos de tiempo, deadlines y cancelación
 - 3. Retries: repetir solo cuando es seguro y útil
 - 4. Circuit Breaker: dejar de insistir
 - 5. Bulkhead: aislar la capacidad
 - 6. Fallback y degradación controlada
 - 7. Componer políticas
 - 8. Resiliencia de extremo a extremo y experiencia controlada
 - 9. Trabajo asíncrono y DLQ
 - 10. Verificar y observar la resiliencia
 - 11. Casos reales
 - 12. Diseño de referencia: una compra cuando Pagos se degrada
 - 13. Trade-offs y criterios de decisión
 - 14. Puntos clave
 - 15. Referencias

Clase 07 — Resiliencia ante fallos parciales

ARQ 2026 · UCC · 23 de septiembre de 2026

1. Contexto: del fallo local a la falla en cascada

La plataforma ya separó Catálogo, Búsqueda, Carrito, Órdenes y Pagos. También dispone de un gateway, balanceo y descubrimiento de servicios. Esa distribución permite que cada capacidad evolucione y escale por separado, pero agrega redes, procesos y proveedores que pueden fallar de manera independiente.

Un sistema distribuido rara vez está simplemente «encendido» o «apagado». Catálogo puede responder mientras Pagos demora; una réplica puede estar sana mientras otra perdió conexión con la base; una operación puede haberse ejecutado aunque su respuesta nunca haya llegado. La pregunta de esta clase no es cómo evitar toda falla, sino cómo impedir que una falla parcial derribe capacidades que todavía podrían entregar valor.

Conviene distinguir tres ideas:

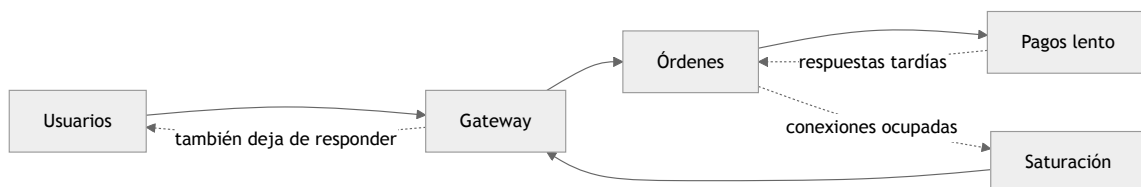
- **Disponibilidad:** el sistema puede atender una operación en un momento dado.

- **Confiabilidad:** mantiene el comportamiento correcto durante un período y bajo las condiciones acordadas.
- **Resiliencia:** contiene una perturbación, conserva las capacidades posibles y recupera su funcionamiento sin ampliar innecesariamente el daño.

También es útil separar *fault* de *failure*. Un **fault** es una condición anómala: una conexión se corta, una réplica responde lento o un proveedor devuelve un error. Se convierte en una **failure** observable cuando el sistema deja de cumplir el resultado comprometido. Si Búsqueda falla pero la compra continúa, el fault fue contenido. Si esa lentitud ocupa todos los recursos de Órdenes y bloquea el checkout, produjo una falla en cascada.

1.1 Por qué la lentitud puede ser peor que un error

Un error inmediato libera recursos y permite tomar otra decisión. Una llamada que no responde conserva conexiones, memoria, goroutines, threads y lugares en colas mientras el llamador espera. Si llegan nuevas requests, aumenta el trabajo en vuelo; la cola crece, la latencia sube y más clientes alcanzan sus límites y reintentan. Una dependencia lenta termina saturando también a sus consumidores.



Google documenta esta dinámica: cuando se pierden deadlines, el trabajo tardío se desperdicia y los retries agregan todavía más carga; además, colas, threads, memoria y file descriptors pueden agotarse hasta iniciar una cascada. El post histórico de Netflix sobre tolerancia a fallos describe el mismo problema en su API: una dependencia con latencia alta podía consumir los threads disponibles y afectar operaciones independientes.

Idea central: resiliencia no significa «todo siempre funciona». Significa limitar el radio de impacto, preservar trabajo útil, decir la verdad sobre el estado y facilitar la recuperación.

2. Presupuestos de tiempo, deadlines y cancelación

Una llamada remota necesita un límite. Sin él, el llamador no sabe cuándo abandonar y puede conservar recursos indefinidamente. Sin embargo, «poner un timeout» no es una política completa.

2.1 Distintos límites para distintas etapas

Según el cliente y el protocolo pueden existir límites diferentes:

- **Timeout de conexión:** cuánto esperar para establecer el canal.
- **Timeout de lectura o respuesta:** cuánto esperar por datos una vez conectados.

- **Timeout de inactividad:** cuánto tolerar sin progreso observable.
- **Deadline:** instante absoluto o presupuesto máximo para completar toda la operación.

El deadline responde a una pregunta de producto: ¿hasta cuándo este resultado todavía es útil? Si el frontend espera 2 segundos, no sirve que Órdenes espere 5 segundos a Pagos. Debe conservar tiempo para interpretar la respuesta y contestar.

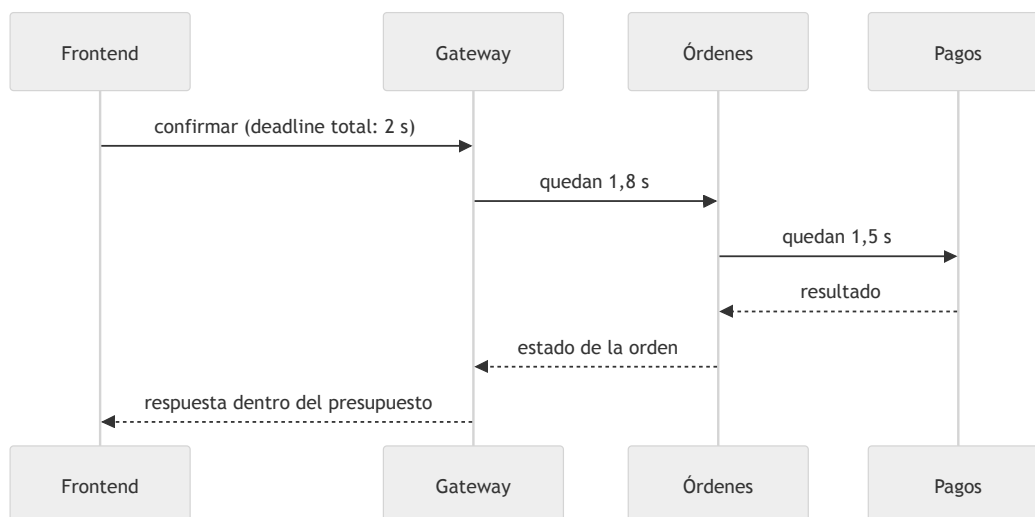
Supongamos este presupuesto ilustrativo:

Etapa	Presupuesto máximo
Gateway y autenticación	100 ms
Órdenes: validación local	150 ms
Pagos, incluidos intentos	1.300 ms
Construcción y retorno de respuesta	250 ms
Margen	200 ms
Total	2.000 ms

No son valores universales. Se eligen a partir del objetivo de experiencia y de distribuciones reales de latencia. Un timeout demasiado largo sostiene la saturación; uno demasiado corto convierte respuestas válidas en errores y puede disparar retries.

2.2 Propagar el presupuesto

Cada capa debe recibir el deadline restante, no iniciar un reloj nuevo. Si el gateway consumió 300 ms de un presupuesto de 2 segundos, Órdenes ya no dispone de 2 segundos completos. Google SRE recomienda propagar el mismo deadline absoluto por el árbol de RPCs y comprobar el tiempo restante antes de iniciar cada etapa.



En Go, `context.Context` permite propagar cancelación y deadlines entre funciones. La documentación oficial muestra que un contexto derivado se cancela cuando vence su propio límite o cuando se cancela el contexto padre. Pero esto requiere cooperación: la API llamada debe aceptar el contexto y detener el trabajo.

2.3 Cancelar la espera no deshace el efecto

Si el navegador deja de esperar, solo sabemos eso: **el navegador dejó de esperar**. No sabemos si Pagos recibió la solicitud, si la rechazó o si confirmó el cargo y se perdió la respuesta.

La cancelación puede ahorrar trabajo cuando se propaga y la operación todavía es interrumpible. No es un rollback distribuido. Una transacción ya confirmada o un request aceptado por un proveedor puede continuar aunque el cliente haya cerrado la pantalla.

Esta ambigüedad conduce a una regla fundamental:

Timeout no equivale a fracaso del negocio. Puede significar resultado desconocido.

3. Retries: repetir solo cuando es seguro y útil

Un retry no elimina el fallo: crea otra ejecución. Puede superar un problema transitorio, pero también duplicar un efecto o aumentar la carga sobre un sistema saturado.

3.1 Clasificar antes de repetir

Situación	¿Retry automático?	Motivo
Conexión interrumpida antes de obtener respuesta	Tal vez	Puede ser transitorio, pero el efecto podría haber ocurrido
Respuesta de sobrecarga con orientación de espera	Sí, si queda presupuesto	La condición puede recuperarse; requiere backoff
Payload inválido	No	Repetir los mismos datos no los corrige
Falta de autorización	No	Esperar no cambia los permisos
Medio de pago rechazado	No	Es una decisión de negocio, no una falla transitoria
Lectura temporalmente fallida	Usualmente sí	La repetición no suele producir un nuevo efecto
Creación o cobro	Solo con idempotencia	Existe riesgo de duplicación

No todo error `5xx` debe reintentarse ni todo `4xx` es necesariamente permanente. Importa el contrato: debe diferenciar con claridad los fallos repetibles de los que exigen cambiar datos, credenciales o decisión de negocio.

3.2 Idempotencia e idempotency keys

Una operación es **idempotente** cuando ejecutarla varias veces deja el mismo efecto observable que ejecutarla una sola vez. Leer una orden por identificador suele ser idempotente. Crear un pago no lo es automáticamente.

Una **idempotency key** convierte varios intentos de una misma intención en una única operación lógica:

1. El cliente crea una clave estable para «pagar la orden 845».
2. Envía la mutación con esa clave.
3. El servidor asocia clave, parámetros y resultado.
4. Si llega otra request con la misma clave y los mismos parámetros, devuelve el resultado ya registrado o informa que sigue en proceso.
5. Si la clave se reutiliza con otros parámetros, rechaza el uso inconsistente.

Stripe documenta este enfoque para repetir creaciones o actualizaciones sin ejecutar dos veces el efecto. Su implementación guarda el status y el cuerpo del primer resultado para una clave y compara los parámetros de usos posteriores. Ese comportamiento es una decisión concreta de Stripe; nuestra arquitectura debe definir explícitamente el suyo.

La clave por sí sola no alcanza. Hay que decidir:

- quién la genera y cuál es su alcance;
- cuánto tiempo se conserva;
- qué ocurre si llega un segundo intento mientras el primero continúa;
- si los parámetros deben coincidir exactamente;
- cómo se persisten resultado y estado;
- qué respuesta obtiene el cliente ante `procesando`, `confirmado` o `rechazado`.

Deshabilitar el botón del frontend reduce dobles clics, pero no protege contra reconexiones, dos dispositivos o retries de infraestructura. La garantía vive en el contrato y en el servidor.

3.3 Backoff, jitter y límites

Reintentar inmediatamente puede golpear una dependencia antes de que se recupere. El **backoff exponencial** aumenta la demora entre intentos:

```
intento 1: inmediato
intento 2: espera base
intento 3: espera base × 2
intento 4: espera base × 4
```

Conviene aplicar un máximo para que la espera no crezca sin control. Aun así, si miles de clientes fallan juntos y usan la misma secuencia, vuelven juntos. El **jitter** agrega variación aleatoria y distribuye esos retries en el tiempo.

Amazon señala que los retries son «egoístas»: consumen más recursos del servidor para aumentar la probabilidad de éxito del cliente. Su recomendación combina timeout, cantidad limitada de retries, backoff con tope y jitter. Google SRE añade un **retry budget**: limitar el tráfico adicional permitido para retries respecto del tráfico original.

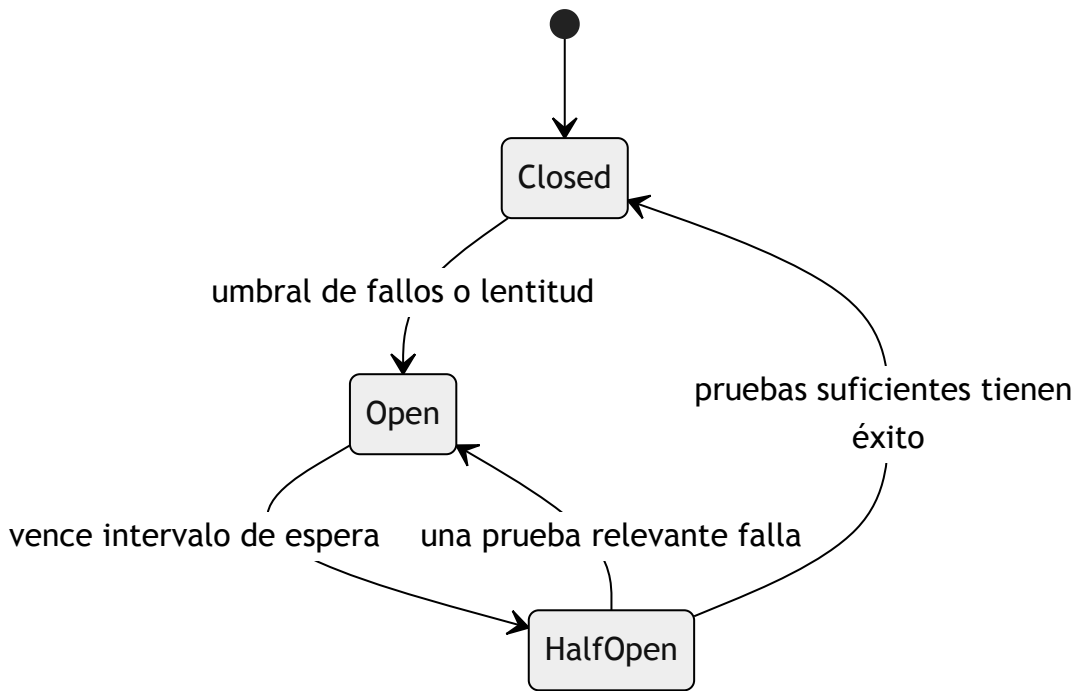
3.4 Amplificación entre capas

Si navegador, gateway y Órdenes hacen hasta 3 intentos cada uno, una sola acción puede generar hasta $3 \times 3 \times 3 = 27$ llamadas a la dependencia inferior. Con 4 intentos por capa serían 64. Por eso se asigna un dueño del retry, normalmente la capa inmediatamente superior a la dependencia y con contexto suficiente para decidir.

***Regla:** un retry necesita error clasificable, tiempo restante, límite, dispersión y una semántica segura. Si falta alguno, puede empeorar el incidente.*

4. Circuit Breaker: dejar de insistir

Cuando una dependencia viene fallando, cada nueva llamada probablemente repetirá una espera costosa. El **Circuit Breaker** observa resultados recientes y, al superar una política, abre el circuito para rechazar rápidamente nuevas ejecuciones.



- **Closed:** deja pasar tráfico y registra éxitos, errores, timeouts y llamadas lentas.
- **Open:** no invoca la dependencia; falla rápido o deriva a una respuesta alternativa.
- **Half-open:** permite pocas pruebas controladas para evaluar recuperación.

4.1 La política importa más que el dibujo

Un circuito necesita una ventana y un volumen mínimo. Con dos requests, un error representa 50 %, pero no hay evidencia suficiente para declarar caída una dependencia. También necesita decidir qué cuenta como fallo: una validación rechazada no indica mala salud; un timeout o una llamada extremadamente lenta sí puede hacerlo.

El alcance tampoco debe ser arbitrario. Un único circuito para todo Pagos puede bloquear consultar un pago porque otra operación distinta, como crear una devolución, falla. Separar por operación ayuda si sus modos de falla son diferentes; crear un circuito por cada URL o instancia puede fragmentar demasiado las señales.

Cada réplica del llamador suele mantener su circuito local y observar una muestra diferente. Coordinar el estado global agrega dependencia y complejidad; no es automáticamente superior.

4.2 Qué no resuelve

El circuit breaker:

- no cancela llamadas que ya están ejecutándose;
- no reemplaza al timeout;
- no impide que otros consumidores sigan saturando la dependencia;
- no vuelve seguro un retry;

- no garantiza que exista un fallback correcto;
- no reconcilia operaciones cuyo resultado quedó incierto.

Su valor es cambiar una espera probablemente inútil por una decisión rápida y medible.

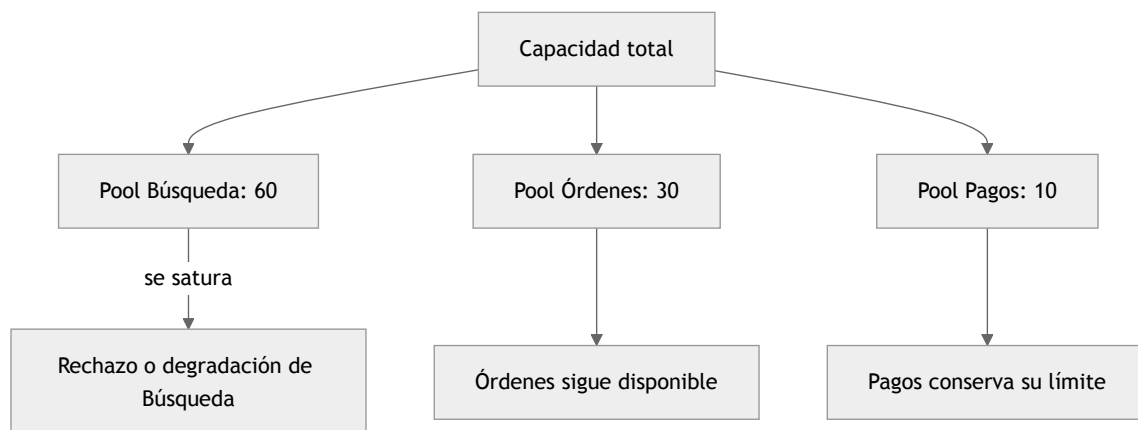
5. Bulkhead: aislar la capacidad

Un barco divide su casco en compartimentos para que una filtración no inunde todo. El patrón **Bulkhead** separa recursos de modo que la saturación de una capacidad no consuma todos los recursos compartidos.

Se puede aislar mediante:

- límites de concurrencia por dependencia u operación;
- pools separados de conexiones, goroutines, threads o workers;
- colas acotadas;
- capacidad reservada para recorridos críticos.

Si Búsqueda se vuelve lenta, sus requests no deberían ocupar todas las conexiones necesarias para consultar Órdenes. Si Pagos admite 40 operaciones simultáneas y ya están ocupadas, aceptar 2.000 más en una cola no crea capacidad: aumenta memoria y hace que esas operaciones venzan antes de ejecutarse.



5.1 Bulkhead no es rate limiting

El **rate limit** decide cuántas requests admitir por unidad de tiempo. El bulkhead decide cuánto trabajo simultáneo puede ocupar un compartimento. Cien requests por segundo que duran 5 ms y cien que duran 5 segundos tienen la misma tasa, pero una concurrencia radicalmente distinta.

El aislamiento paga un costo: una partición puede estar ociosa mientras otra rechaza tráfico. Compartir todo maximiza utilización en el caso sano; separar limita el radio de impacto en el caso degradado. La decisión depende de qué operaciones merecen conservar capacidad.

6. Fallback y degradación controlada

Un **fallback** ofrece una respuesta alternativa cuando la principal no está disponible. No consiste en convertir cualquier error en `200 OK`; debe conservar una semántica útil y honesta.

Capacidad	Degradación posible	Degradación peligrosa
Recomendaciones	Omitir el carrusel	Inventar recomendaciones como si fueran personalizadas
Búsqueda	Reducir señales costosas o mostrar resultados cacheados con aviso	Confirmar stock desde un índice atrasado
Catálogo	Mostrar datos anteriores con su timestamp	Presentarlos como precio final actual
Pagos	Informar <code>verificando</code> y permitir consultar estado	Aprobar o rechazar sin conocer el resultado
Fraude/autorización	Rechazar o llevar a revisión según riesgo	Permitir automáticamente una operación crítica

Las respuestas pueden ser datos cacheados, funcionalidad reducida, procesamiento diferido, consulta posterior o rechazo explícito. Lo decisivo es qué promesa de negocio se mantiene.

6.1 Fail-open y fail-closed

Ante la indisponibilidad de un control, **fail-open** permite continuar y **fail-closed** bloquea. Ninguna opción es universal:

- Si no carga una recomendación, omitirla conserva el checkout.
- Si no puede verificarse autorización, permitir por defecto puede exponer recursos.
- Si fraude no responde, el negocio puede rechazar, dejar pendiente o aceptar bajo límites de riesgo previamente acordados.

La elección combina impacto económico, seguridad, reversibilidad y experiencia. Debe ser explícita, no el efecto accidental de capturar una excepción.

6.2 El fallback también puede fallar

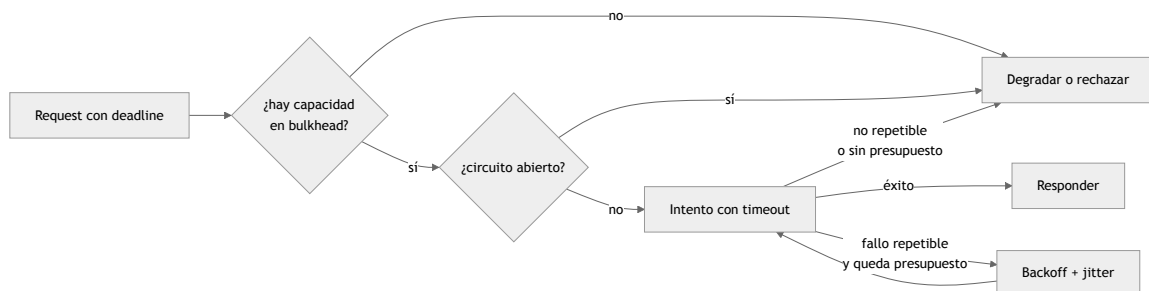
Un fallback que consulta otra dependencia remota puede sufrir la misma saturación o crear un camino circular. Netflix recomendaba fallbacks simples, como caché local o datos estáticos, y aislaba los que podían consumir recursos. Toda alternativa necesita condiciones de activación, vigencia, límites y métricas.

7. Componer políticas

Los patrones no son casilleros independientes. Una política típica contiene:

1. un deadline extremo a extremo;
2. límites por intento;
3. retries acotados cuando son seguros;
4. un circuito que evita insistencia sostenida;
5. un bulkhead que limita los recursos en juego;
6. una respuesta degradada o un error honesto.

Esto no define un orden universal de decoradores. Si el timeout envuelve cada intento, dos intentos pueden superar el tiempo total esperado. Si envuelve toda la política, el segundo intento dispone solo del remanente. Si el circuito observa cada intento, registra más fallos que si observa el resultado lógico final. Ambas elecciones pueden ser correctas, pero deben corresponder a lo que se quiere medir y limitar.



Estas políticas también interactúan con el balanceo de tráfico entre réplicas. Cambiar de réplica puede ayudar cuando el fallo es propio de esa instancia, pero reintentar contra todas las réplicas durante una degradación compartida —por ejemplo, una dependencia común saturada detrás de todas ellas— no reparte la carga: la multiplica sobre ese mismo recurso crítico.

7.1 Antipatrones

- reintentar en cada capa;
- usar el mismo timeout para toda operación;
- encolar sin límite;
- tratar toda excepción como transitoria;
- enviar el fallback a la misma dependencia caída;
- compartir un circuito entre operaciones con modos de falla distintos;
- ocultar fallos con respuestas vacías exitosas;
- copiar umbrales de otro servicio sin observar latencia ni capacidad propias.

8. Resiliencia de extremo a extremo y experiencia controlada

La resiliencia no termina en el backend. El frontend debe traducir el estado técnico a una experiencia que permita entender, esperar o recuperarse sin provocar un segundo daño.

8.1 Estados de negocio, no solo estados HTTP

Para una compra conviene distinguir al menos:

```
procesando → confirmada
           → rechazada
           → verificando / resultado desconocido
```

- **Procesando:** el sistema aceptó la intención y todavía trabaja.
- **Confirmada:** existe evidencia durable del resultado exitoso.
- **Rechazada:** existe una decisión definitiva que permite corregir o intentar otra alternativa.
- **Verificando:** la respuesta se perdió o una dependencia no permite determinar aún el resultado.

Un spinner infinito no es un estado de negocio. Tampoco lo es mostrar «hubo un error» sin explicar si el usuario debe esperar, consultar o repetir.

8.2 Flujo recuperable

1. El frontend genera o recibe un identificador estable de la intención.
2. Envía la confirmación una sola vez desde la interacción, con protección idempotente.
3. Si vence la espera, no declara rechazo: conserva la referencia.
4. Consulta el estado de esa misma operación.
5. Solo habilita una intención nueva cuando la anterior terminó o existe una decisión explícita para reemplazarla.

Cancelar una request al cambiar de pantalla evita consumir una respuesta que ya no se usará, pero no prueba que el servidor detuvo el efecto. Preservar el carrito, la orden y el identificador permite continuar después sin reiniciar todo el recorrido.

8.3 Degradación visible

Si se muestran datos anteriores, debe informarse su contexto temporal. Si una capacidad no esencial se retira, el recorrido principal debe seguir siendo comprensible. No se trata de exhibir detalles de infraestructura, sino de comunicar la decisión relevante: «estamos verificando el pago» es mejor que «timeout en servicio X» y mucho más correcto que «pago rechazado» cuando el resultado se desconoce.

Este enfoque no depende de React, Axios ni una librería particular. Es un contrato de extremo a extremo entre experiencia, API y dominio.

9. Trabajo asíncrono y DLQ

Una request síncrona debe responder, degradar o cortar dentro de un presupuesto. Un mensaje durable puede esperar y reprocesarse sin mantener abierta la interacción original.

El circuit breaker protege a un consumidor cuando su dependencia está degradada. No decide qué hacer con un mensaje que nunca podrá procesarse. Después de clasificar el error, aplicar backoff y agotar intentos, una **Dead Letter Queue** puede aislar ese mensaje para investigarlo y recuperarlo de forma controlada.

Mecanismo	Protege frente a	No reemplaza
Timeout	Espera ilimitada	Idempotencia ni cancelación del efecto
Circuit breaker	Insistencia síncrona probablemente inútil	DLQ ni reconciliación
Bulkhead	Saturación de recursos compartidos	Control del resultado de negocio
DLQ	Mensaje durable que no pudo completarse	Contención de llamadas síncronas

Una DLQ necesita dueño, alertas, retención y procedimiento de reproceso propios. No es una papelera ni un sustituto de la resiliencia transversal que describe este apunte.

10. Verificar y observar la resiliencia

Una política no funciona porque exista en configuración. Hay que demostrar que limita el daño y que el sistema recupera la capacidad esperada.

10.1 Pruebas por nivel

- **Componente:** simular error transitorio, timeout, respuesta lenta y saturación; verificar intentos, deadline, apertura del circuito y ausencia de duplicados.
- **Integración:** introducir latencia o indisponibilidad en una dependencia y comprobar que operaciones independientes continúan.
- **Extremo a extremo:** perder la respuesta después de confirmar un efecto y verificar que la experiencia muestra `verificando`, consulta el mismo identificador y no duplica la operación.

Señales mínimas:

- tasa de timeouts y retries;
- intentos por request original;
- circuitos abiertos, duración y pruebas half-open;

- rechazos por límite de concurrencia;
- uso y resultado de fallbacks;
- cantidad y antigüedad de resultados desconocidos;
- operaciones conciliadas y mensajes en DLQ.

Que el circuito vuelva a cerrar indica recuperación técnica de la llamada. No demuestra que las órdenes inciertas hayan sido conciliadas.

10.2 Chaos Engineering, con límites

Chaos Engineering formula una hipótesis sobre el comportamiento estable, introduce una perturbación acotada y observa si el sistema mantiene esa propiedad. No significa «romper producción para ver qué pasa».

Antes de experimentar hacen falta pruebas convencionales, observabilidad, un radio de impacto controlado y una condición de detención. Un primer ejercicio puede ejecutarse en integración: «si Pagos tarda más que el presupuesto, Búsqueda y consulta de Órdenes continúan, el checkout informa estado incierto y no aparecen cobros duplicados».

La instrumentación general, los SLOs, las alertas y la operación en producción exceden el alcance de este apunte: aquí solo se cubren las señales necesarias para razonar sobre estas políticas de resiliencia.

11. Casos reales

11.1 Amazon: timeouts, retries y backoff con jitter como práctica de ingeniería

Amazon documenta en su Builders' Library cómo diseña llamadas remotas dentro de AWS: cada cliente debe fijar un timeout acotado, limitar la cantidad de reintentos, espaciarlos con backoff exponencial y sumar jitter para evitar que muchos clientes vuelvan a intentar en el mismo instante. La guía es explícita sobre el costo: un retry consume recursos del servidor que recibe la llamada para mejorar la probabilidad de éxito de quien reintenta. Por eso necesita límites de intentos y de tráfico adicional; Google SRE formaliza este último control mediante *retry budgets*. Un segundo documento de la misma serie extiende el problema a las mutaciones: sin una API idempotente, un cliente que reintenta después de perder una respuesta no puede saber si repite una operación ya aplicada o crea una nueva. [Amazon Builders' Library — Timeouts, retries, and backoff with jitter](#); [Amazon Builders' Library — Making retries safe with idempotent APIs](#); [Google SRE Book — Handling Overload](#).

El caso no describe un incidente puntual, sino la práctica de ingeniería que sostiene las secciones 2 y 3 de este apunte: timeout, retry, backoff, jitter e idempotencia como piezas de una misma decisión, no como configuraciones aisladas.

11.2 Netflix: el aislamiento de dependencias que dio origen a Hystrix

En 2012, Netflix describió cómo su API componía respuestas llamando a decenas de subsistemas. Una dependencia lenta podía bloquear threads del contenedor hasta afectar toda la API. Su `DependencyCommand`,

precursor del enfoque popularizado por Hystrix, combinó timeouts y retries, pools por dependencia, semáforos, circuit breakers y fallback. [Netflix Technology Blog — Fault Tolerance in a High Volume, Distributed System](#); [Netflix/Hystrix — How It Works](#).

La decisión importante no fue una biblioteca particular, sino el mecanismo:

- una dependencia lenta solo podía ocupar su pool;
- cuando no había capacidad se rechazaba pronto en lugar de acumular trabajo;
- el circuito liberaba presión durante una degradación sostenida;
- el fallback intentaba conservar una experiencia útil;
- métricas diferenciaban error, timeout, rechazo y circuito abierto.

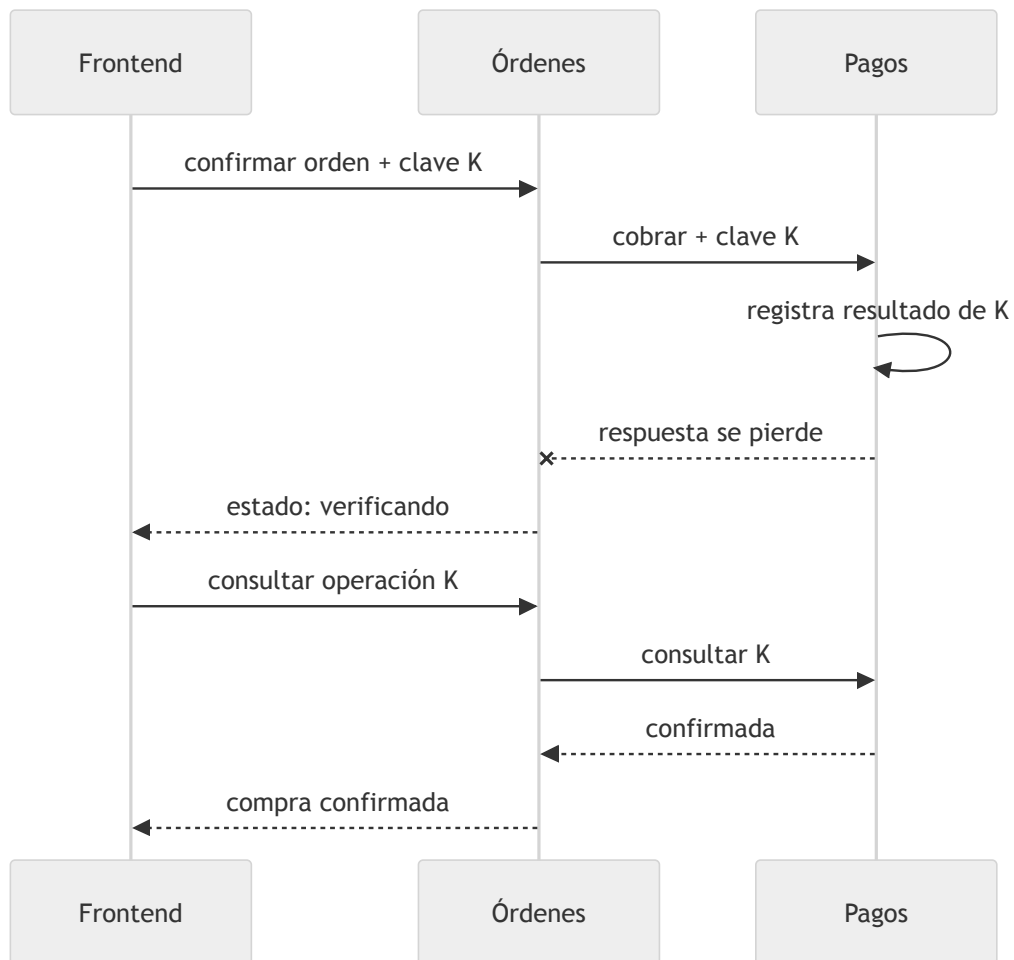
Hystrix hoy está en modo de mantenimiento y no se propone como elección tecnológica. [Netflix/Hystrix — repositorio y estado de mantenimiento](#). El caso sigue siendo valioso porque muestra cómo timeout, circuit breaker, bulkhead y fallback se diseñan como una política, no como anotaciones independientes.

11.3 Stripe: idempotency keys como contrato para no duplicar un cobro

Stripe expone en su API pública el mecanismo de idempotency keys descrito en la sección 3.2: el cliente genera una clave única por intención de pago y la envía en un header en cada reintento de esa misma operación. Según la documentación de Stripe, el servidor guarda el resultado de la primera ejecución asociado a esa clave; si llega una segunda request con la misma clave y los mismos parámetros, devuelve el resultado ya registrado en lugar de ejecutar el cobro de nuevo, y si los parámetros no coinciden, rechaza la request. El caso muestra la decisión concreta de un proveedor de pagos real frente al mismo problema que la sección 3.2 plantea en abstracto: una red que puede perder la respuesta sin que el cliente sepa si el efecto ya ocurrió. [Stripe API Reference — Idempotent requests](#).

12. Diseño de referencia: una compra cuando Pagos se degrada

Una persona confirma una compra. La respuesta de Pagos se pierde después de que el proveedor pudo haber registrado el cargo.



Decisiones:

1. **Deadline:** Órdenes reserva margen para responder antes del límite del frontend.
2. **Idempotencia:** todos los reenvíos de la misma intención conservan la clave `K`.
3. **Retry:** solo se repiten fallos transitorios, con backoff y mientras queda presupuesto.
4. **Circuit breaker:** una degradación sostenida de crear pagos falla rápido; consultar estado puede tener otro circuito si su comportamiento es independiente.
5. **Bulkhead:** la espera de Pagos no consume recursos reservados para consultar órdenes o navegar Catálogo.
6. **Fallback:** nunca «aprueba» el cobro; devuelve `verificando` cuando el resultado es incierto.
7. **Experiencia:** el frontend conserva orden y clave, evita una nueva intención y ofrece consultar estado.
8. **Recuperación asíncrona:** una conciliación posterior puede completar el estado; si su mensaje agota intentos, la DLQ conserva el caso para intervención.

El objetivo no es que el fallo sea invisible. Es que el usuario no pague dos veces, que la plataforma preserve las capacidades sanas y que exista un camino claro hasta una resolución definitiva.

13. Trade-offs y criterios de decisión

Decisión	Conviene cuando	Gana	Paga
Timeout/deadline	Toda llamada remota o trabajo con utilidad temporal	Recursos acotados y respuesta previsible	Puede cortar trabajo que habría terminado
Retry	Fallo transitorio, operación segura y presupuesto restante	Supera fallos breves	Agrega latencia y carga
Idempotency key	Una mutación puede repetirse por incertidumbre de red	Evita efectos duplicados	Persistencia, expiración y estados intermedios
Circuit breaker	Fallar rápido protege recursos durante degradación sostenida	Reduce espera e insistencia	Umbrales, estado local y falsos positivos
Bulkhead	Varias capacidades comparten recursos agotables	Reduce radio de impacto	Menor utilización conjunta
Fallback	Existe un resultado alternativo honesto y útil	Conserva valor para el usuario	Menor calidad o datos anteriores
DLQ	Trabajo durable no debe bloquear el flujo tras agotar su política	Aísla y preserva evidencia	Exige operación y recuperación

Antes de introducir un patrón deben responderse cuatro preguntas:

1. ¿Qué modo de falla concreto reduce?
2. ¿Qué recurso o promesa protege?
3. ¿Qué nueva falla puede crear?
4. ¿Cómo sabremos que funcionó y que se recuperó?

14. Puntos clave

- La lentitud retiene recursos y puede iniciar una cascada aun sin errores explícitos.
- Un timeout limita la espera; no demuestra que el efecto remoto no ocurrió.
- Los deadlines se propagan: cada capa trabaja con el presupuesto restante.
- Retry requiere clasificación, límite, backoff, jitter e idempotencia cuando hay efectos.
- Reintentar en varias capas multiplica carga; la política necesita un dueño.

- Circuit breaker falla rápido frente a una dependencia degradada, pero no la repara.
 - Bulkhead limita concurrencia y radio de impacto; no es lo mismo que rate limiting.
 - Un fallback debe ser semánticamente honesto. No se inventan stock, autorización ni pagos.
 - El frontend representa estados de negocio, incluido el resultado desconocido, y conserva una vía de recuperación.
 - DLQ atiende trabajo durable que no pudo completarse; circuit breaker contiene insistencia síncrona.
 - Resiliencia se valida provocando fallos acotados y observando contención, experiencia y recuperación.
-

15. Referencias

- [Amazon Builders' Library — Timeouts, retries, and backoff with jitter.](#)
- [Amazon Builders' Library — Making retries safe with idempotent APIs.](#)
- [Google SRE Book — Addressing Cascading Failures.](#)
- [Google SRE Book — Handling Overload.](#)
- [Netflix Technology Blog — Fault Tolerance in a High Volume, Distributed System.](#)
- [Netflix/Hystrix — How It Works.](#)
- [Netflix/Hystrix — repositorio y estado de mantenimiento.](#)
- [Stripe API Reference — Idempotent requests.](#)
- [Go Documentation — Canceling in-progress operations.](#)
- [Principles of Chaos Engineering.](#)